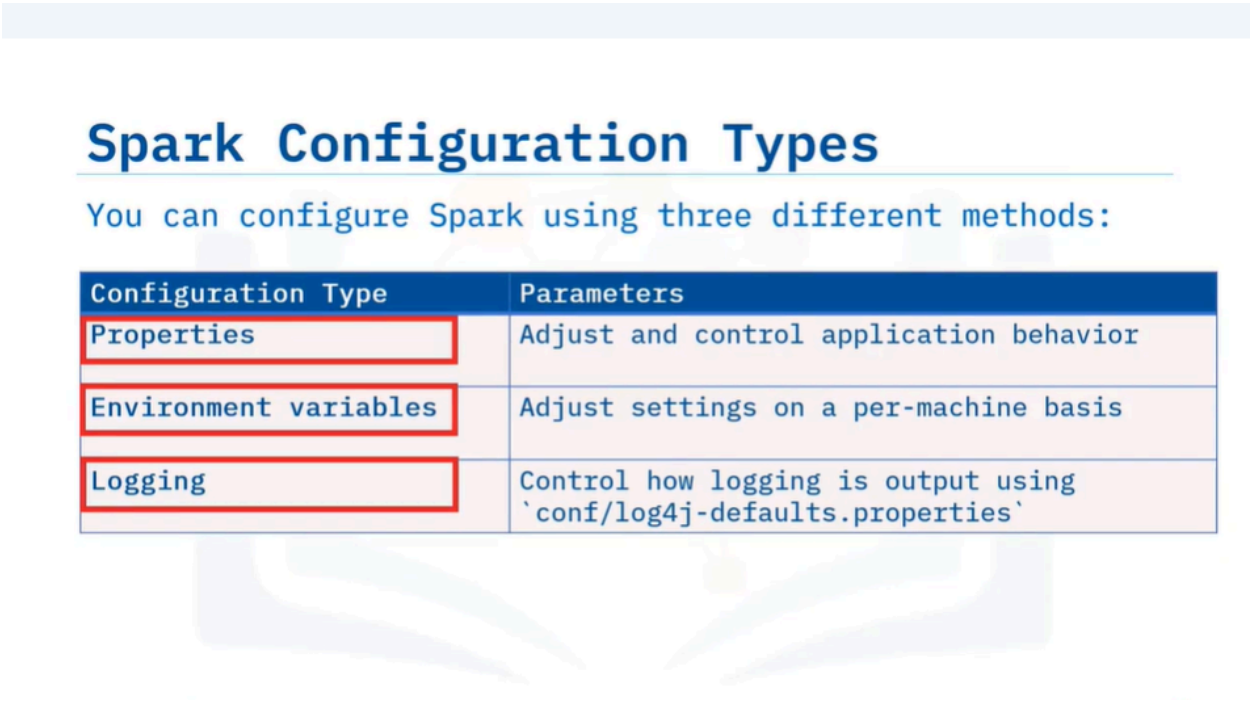
Setting Apache Spark Configuration



Spark Configuration Types

You can configure Spark using three different methods:

Configuration Type	Parameters
Properties	Adjust and control application behavior
Environment variables	Adjust settings on a per-machine basis
Logging	Control how logging is output using `conf/log4j-defaults.properties`

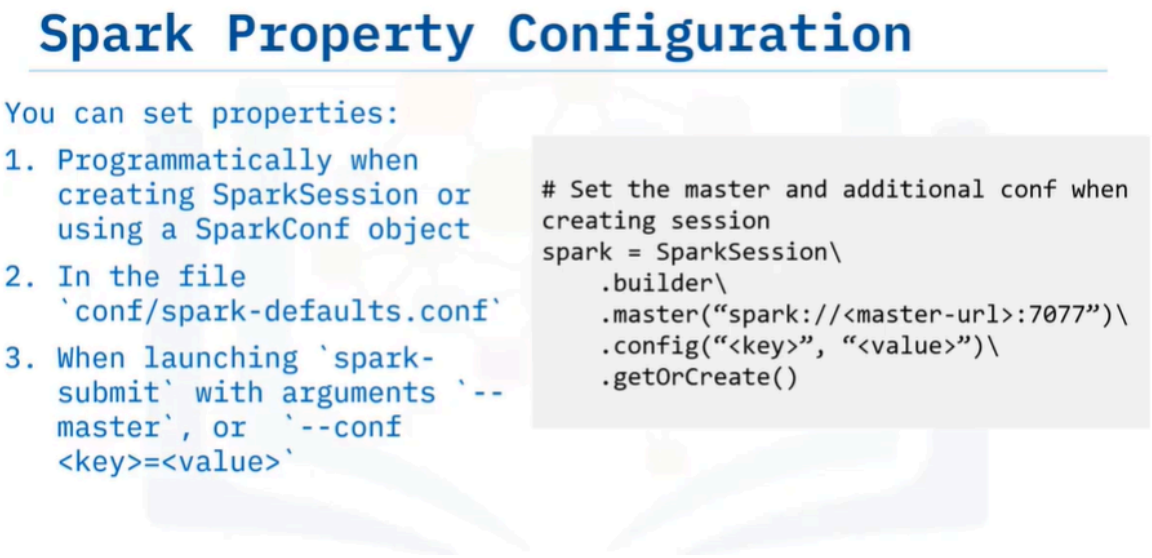
- You can configure a Spark Application using three different methods:
  - properties, environment variables, or logging configuration.
  - You can set Spark properties to adjust and control most application behaviors, including setting properties with the driver and sharing them with the cluster.
  - Environment variables are loaded on each machine, so they can be adjusted on a per-machine basis if hardware or installed software differs between the cluster nodes.
  - Spark logging is controlled by the log4j defaults file, which dictates what level of messages, such as info or errors, are logged to file or output to the driver during application execution.

Spark Configuration Location

- Configuration files are located under the `conf/` directory in the Spark installation
- Files are not created by default, however Spark provides template files that can be renamed as shown in the table

Configuration Type	Template File	Actual File
Spark properties	spark-defaults.conf.template	spark-defaults.conf
Environment variables	spark-env.sh.template	spark-env.sh
Logging properties	log4j.properties.template	log4j.properties

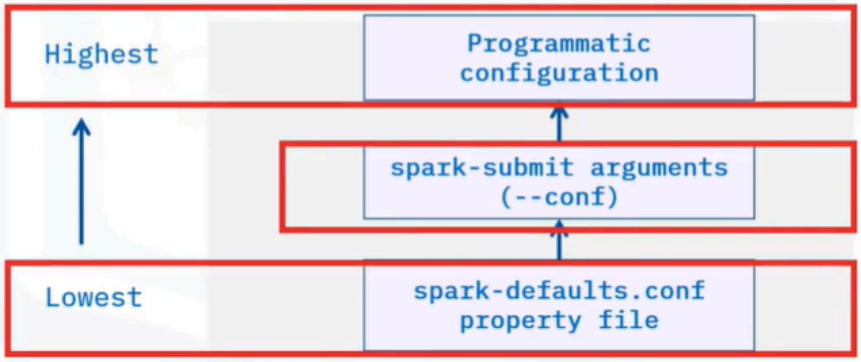
- Spark configuration files are located under the “conf” directory in the installation. By default, there are no preexisting files after installation, however Spark provides a template for each configuration type with the filenames shown here. You can create the appropriate file by removing the ‘.template’ extension. Inside the template files, are sample configurations for common settings. You can enable them by uncommenting.



- You can configure Spark properties in a few different ways. In the driver program, you can set the configuration programmatically when creating the SparkSession or by using a separate SparkConf object that is then passed into the session constructor. Properties can be set in a configuration file found at 'conf/spark-defaults.conf'.
- Properties can also be set when launching the application with spark-submit, either by using provided options such as '--master' or using the "--conf" option with a key, value pair.

Spark Property Precedence

Spark properties use the following precedence and are merged into a final configuration before running the application



- Because Spark properties can be set in different ways, let's look at how they are merged to build the final configuration for the application. The configuration set programmatically takes the highest precedence. This means that if a configuration has already been set elsewhere or programmed into the application it will be overwritten. Next is the configuration provided with the spark-submit script. Last is any configuration set in the spark-defaults.conf file.

Where to Use Static Configuration

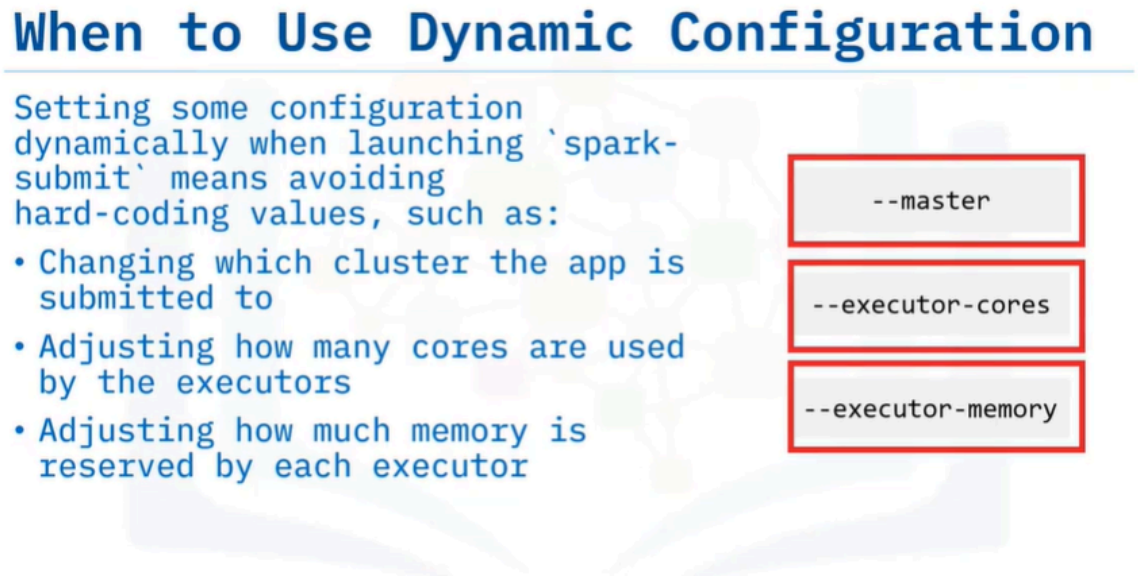
Set static configuration programmatically inside the application for:

- Values that do not change from run to run
- Properties related to the application, not deployment

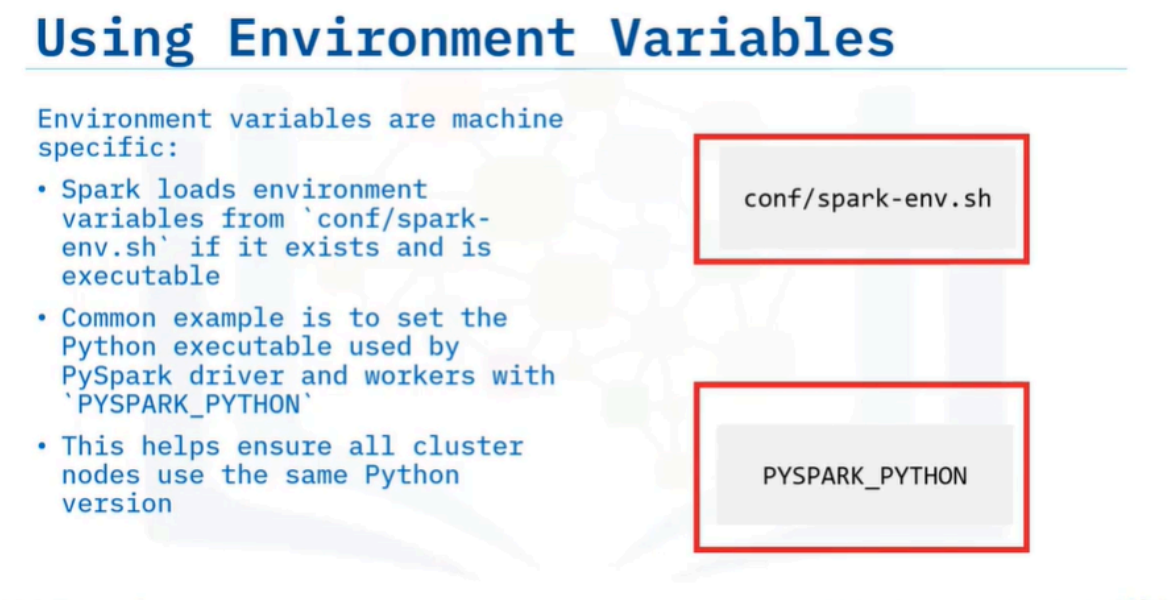
For example, application name does not change if running in cluster versus local mode

```
spark = SparkSession\
    .builder\
    .appName("MySparkApplication")\
    .config("spark.driver.maxResultSize", "2g")\
    .getOrCreate()
```

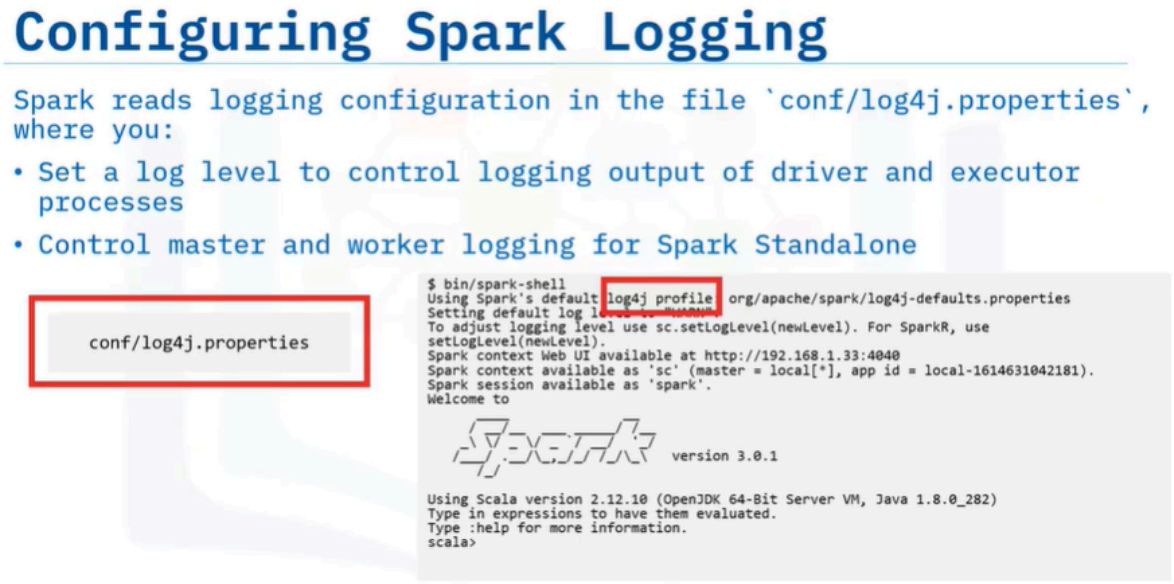
- Static configuration refers to settings that are written programmatically into the application itself. These settings are not usually changed because it requires modifying the application itself. Use static configuration for something that is unlikely to be changed or tweaked between application runs, such as application name or other properties related to the application only. For example, you would not use static configuration when it depends on what type of cluster or resource requirements are available. The example here sets the application name property and there is an additional configuration to set the maximum result size in the driver to 2 gigabytes.



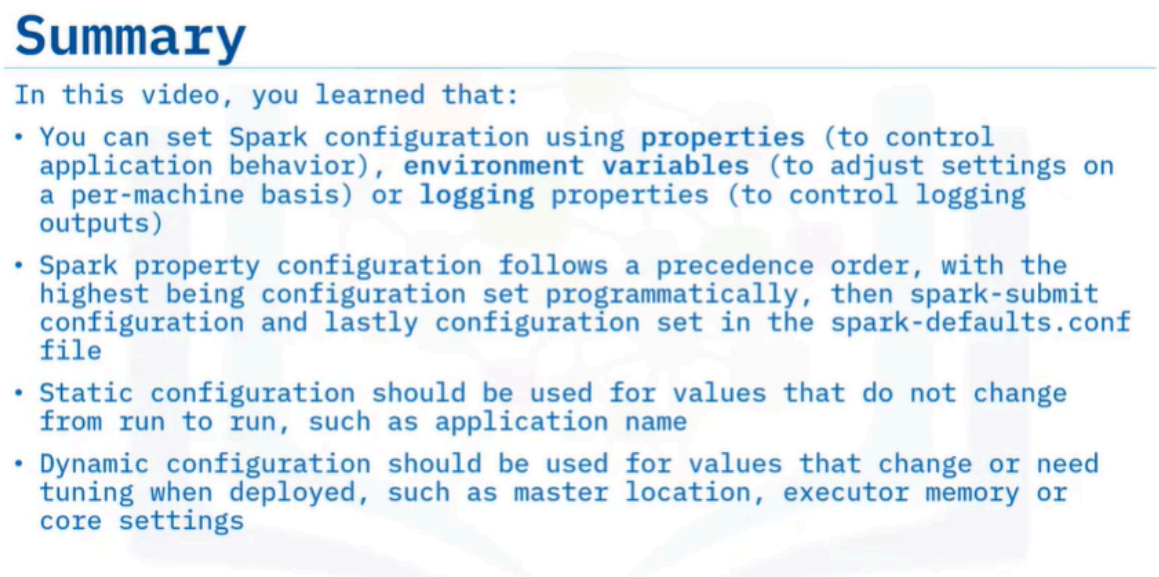
- Spark dynamic configuration is useful to avoid hardcoding specific values in the application itself. This is usually done for configuration such as the master location, so that the application can be launched on a cluster by simply changing the master location. Other examples include setting dynamically how many cores are used or how much memory is reserved by each executor so that they can be properly tuned for whatever cluster the application is run on.



- Spark environment variables are loaded from the file ‘conf/spark-env.sh’.
- These are loaded from each machine in the cluster when a Spark process is started. Since these can be loaded differently for each machine, it can help configure specifics on a per-machine basis. A common usage is to ensure each machine in the cluster uses the same Python executable by setting the “PYSPARK_PYTHON” environment variable.



- Spark logging is controlled using log4j and the configuration is read through
- 'conf/log4j-properties'. Here you can adjust a log level to determine which messages (such as debug, info or errors) are shown in the Spark logs. Log4j allows the configuration to set where the logs are sent to and adjust specific components of Spark or third-party libraries.



Sure! Let's talk about **Apache Spark Configuration** in simple terms.

When you're working with Apache Spark, you need to set up some rules or settings that tell Spark how to behave while running your application. Think of it like preparing a recipe: you need to gather your ingredients and decide how to cook them. In Spark, you can configure it using three main methods: properties, environment variables, and logging configuration.

- **Properties** are like the specific instructions in your recipe. You can set them in your code, in a special configuration file, or when you start your application. For example, you might set the name of your application or how much memory it can use.
- **Environment variables** are like the kitchen tools you have available. They can be different on each machine in your cluster, so you can adjust them based on what each machine needs. For instance, you might want to ensure that all machines use the same version of Python.
- **Logging configuration** helps you keep track of what's happening while your application runs, like taking notes while cooking to remember what worked well or what didn't.

How can you set properties in Spark?

You can set properties in Spark in several ways:

1. **Programmatically in Code**:
    - When you create a `SparkSession`, you can set properties directly in your code. For example:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .config("spark.executor.memory", "2g") \
    .getOrCreate()
```

2. **Configuration File**:
    - You can set properties in a configuration file named `spark-defaults.conf`, located in the `conf` directory of your Spark installation. You can add lines like:

```
spark.app.name MyApp
spark.executor.memory 2g
```

3. **Using `spark-submit`**:
    - When you launch your application using the `spark-submit` command, you can specify properties with the `-conf` option. For example:

```
spark-submit --conf spark.executor.memory=2g --conf spark.app.name=MyApp my_script.py
```

These methods allow you to control how your Spark application runs by adjusting its settings.

What is the purpose of spark-defaults.conf?

The `spark-defaults.conf` file serves an important purpose in Apache Spark:

- **Default Configuration Settings**: It allows you to set default properties for your Spark applications. This means that any application you run will automatically use these settings unless you override them with other configurations.
- **Centralized Management**: By placing common configurations in this file, you can manage settings in one location rather than specifying them every time you run an application. This is especially useful for settings that are consistent across multiple applications.
- **Example Configurations**: The file can include sample configurations that you can uncomment and modify according to your needs. This helps you understand what settings are available and how to use them.

For example, you might set properties like:

```
spark.app.name MyApp
spark.executor.memory 2g
```

This way, every time you run a Spark application, it will use "MyApp" as the application name and allocate 2 gigabytes of memory to each executor by default.

How can you modify spark-defaults.conf for your application?

To modify the `spark-defaults.conf` file for your application, follow these steps:

1. **Locate the File**:
   - Find the `spark-defaults.conf` file in the `conf` directory of your Spark installation. If it doesn't exist, you can create one by copying the template file `spark-defaults.conf.template`.
2. **Open the File**:
   - Use a text editor (like Notepad, Vim, or any code editor) to open the `spark-defaults.conf` file.
3. **Add or Modify Properties**:
   - You can add new properties or modify existing ones. Each property should be on a new line in the format:

   ```
   property_name property_value
   ```

   For example:

   ```
   spark.app.name MyApplication
   spark.executor.memory 4g
   spark.driver.cores 2
   ```

4. **Save the Changes**:
    - After making your changes, save the file and close the text editor.
5. **Run Your Application**:
    - When you run your Spark application, it will automatically use the properties defined in the `spark-defaults.conf` file.

By modifying this file, you can customize the default settings for your Spark applications without needing to specify them each time you run a job.